AULA 07
FUNÇÕES E MÓDULOS

Programação em Python • Parte 1 de 2
Curso de Capacitação em Desenvolvimento Full Stack
Tema: Funções e Módulos • 8 horas
Professor: Msc. Hygo Sousa de Oliveira

Roteiro da aula

O que vamos ver nesta aula

1. Definindo e chamando funções
2. Retorno e parâmetros: padrão, *args e **kwargs
3. Docstrings, type hints e callbacks
4. Escopo de variáveis e decorators
5. Funções recursivas
6. Exemplos guiados e erros comuns

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Definir funções e usar o retorno**
  return, None e vários valores
- **Usar parâmetros padrão, *args e **kwargs**
  Funções flexíveis e bem documentadas
- **Entender escopo, callbacks e decorators**
  Onde cada variável vive e como passar funções adiante
- **Escrever funções recursivas**
  Caso base, caso recursivo e limite de chamadas

Módulo 4 – Funções e Módulos

- Definição e chamada de funções
- Parâmetros e argumentos
- Escopo de variáveis
- Funções recursivas
- Módulos e bibliotecas padrão (Aula 8)

Definindo e chamando funções

Um bloco de código com nome, reutilizável quantas vezes for preciso

funcao.py

```python
def saudacao(nome):
    print(f"Olá, {nome}!")

saudacao("Ana")
saudacao("Bruno")
```

SAÍDA

```
Olá, Ana!
Olá, Bruno!
```

NOTA `def` define a função e o corpo é indentado. Nada executa até a **chamada** `saudacao(...)`.

return: devolvendo valores

print() mostra na tela; return entrega o valor a quem chamou

retorno.py

```python
def area_retangulo(base, altura):
    return base * altura

def so_imprime(valor):
    print("valor:", valor)

area = area_retangulo(5, 3)
print(area)
resultado = so_imprime(area)
print(resultado)
```

SAÍDA

```
15
valor: 15
None
```

ATENÇÃO Sem `return`, a função devolve `None`. O `return` também encerra a função na mesma hora.

Retornando vários valores

Python devolve uma tupla, e você pode desempacotá-la

varios.py

```
def estatisticas(numeros):
    return min(numeros), max(numeros), sum(numeros) / len(numeros)

menor, maior, media = estatisticas([4, 8, 6])
print(menor, maior, media)
print(estatisticas([4, 8, 6]))
```

SAÍDA

```
4 8 6.0
(4, 8, 6.0)
```

DICA O retorno `a, b, c` é uma tupla. Receba em variáveis separadas ou em uma só.

Parâmetros: posicionais, nomeados e padrão

Parâmetro é o nome na definição; argumento é o valor na chamada

parametros.py

```
def apresentar(nome, cargo="Aluno", turma="A"):
    print(f"{nome} - {cargo} ({turma})")

apresentar("Ana")
apresentar("Bruno", "Monitor")
apresentar(turma="B", nome="Carla")
```

SAÍDA

```
Ana - Aluno (A)
Bruno - Monitor (A)
Carla - Aluno (B)
```

NOTA Parâmetros com valor padrão vêm **depois** dos obrigatórios. Argumentos nomeados podem ir em qualquer ordem.

*args: quantidade variável de argumentos

Os argumentos extras chegam agrupados em uma tupla

args.py

```python
def somar(*numeros):
    return sum(numeros)

print(somar(1, 2, 3))
print(somar(10, 20, 30, 40, 50))
valores = [4, 5, 6]
print(somar(*valores))
```

SAÍDA

```
6
150
15
```

NOTA Na definição, `*numeros` agrupa. Na chamada, `*valores` **desempacota** a lista em argumentos.

**kwargs: argumentos nomeados extras

Os nomeados que sobram chegam em um dicionário

kwargs.py

```
def ficha(nome, **dados):
    print(nome)
    for chave, valor in dados.items():
        print(f"  {chave}: {valor}")

ficha("Ana", idade=28, cidade="Boa Vista")
```

SAÍDA

```
Ana
  idade: 28
  cidade: Boa Vista
```

DICA Ordem na definição: `def f(obrigatorios, padrao=1, *args, **kwargs)`.

Docstrings e type hints

Documentação que acompanha a função

docstring.py

```python
def calcular_imc(peso: float, altura: float) -> float:
    """Calcula o IMC.

    Args:
        peso: peso em kg.
        altura: altura em metros.
    """
    return peso / altura ** 2

print(round(calcular_imc(70, 1.75), 2))
print(calcular_imc.__doc__.splitlines()[0])
```

SAÍDA

```
22.86
Calcula o IMC.
```

NOTA A docstring fica logo abaixo do `def`. Os type hints (`: float`, `-> float`) documentam, mas o Python não os impõe.

Funções como argumento (callbacks)

Funções são valores: podem ser passadas, guardadas e devolvidas

callback.py

```
def aplicar(funcao, valores):
    return [funcao(v) for v in valores]

def dobro(x):
    return x * 2

print(aplicar(dobro, [1, 2, 3]))
print(aplicar(str.upper, ["ana", "bia"]))
print(aplicar(lambda x: x ** 2, [1, 2, 3]))
```

SAÍDA

```
[2, 4, 6]
['ANA', 'BIA']
[1, 4, 9]
```

ATENÇÃO Passe `dobro`, e não `dobro()`: sem parênteses você entrega a função; com parênteses, o resultado.

Escopo de variáveis

Onde cada nome existe e pode ser usado

escopo.py

```
taxa = 0.1
def preco_final(preco):
    desconto = preco * taxa
    return preco - desconto
print(preco_final(200))
print(desconto)
```

SAÍDA

```
180.0
Traceback (most recent call last):
  File "escopo.py", line 6, in <module>
    print(desconto)
          ^^^^^^^^
NameError: name 'desconto' is not defined
```

NOTA `taxa` é global e `desconto` é **local**: não existe fora da função.

Decorators: funções que modificam funções

A base para o padrão Registry das próximas atividades

decorator.py

```python
def com_log(funcao):
    def interna(*args, **kwargs):
        print(f"Chamando {funcao.__name__}")
        return funcao(*args, **kwargs)
    return interna

@com_log
def somar(a, b):
    return a + b

print(somar(2, 3))
```

SAÍDA

```
Chamando somar
5
```

NOTA `@com_log` equivale a `somar = com_log(somar)`. O decorator recebe uma função e devolve outra.

Recursão: a função que chama a si mesma

Todo problema recursivo precisa de um caso base

Caso base
quando parar de chamar

Caso recursivo
chama a si mesma com um problema menor

Convergência
cada chamada chega mais perto do caso base

fatorial.py

```python
def fatorial(n):
    if n <= 1:
        return 1
    return n * fatorial(n - 1)

print(fatorial(5))
```

SAÍDA

```
120
```

Recursão: caso base e limite

Sem caso base, o Python interrompe com RecursionError

fibonacci.py

```python
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print([fib(i) for i in range(8)])
```

SAÍDA

```
[0, 1, 1, 2, 3, 5, 8, 13]
```

sem_base.py

```python
def sem_base(n):
    return sem_base(n + 1)

try:
    sem_base(1)
except RecursionError as erro:
    print(erro)
```

SAÍDA

```
maximum recursion depth exceeded
```

DICA Fibonacci recursivo funciona, mas repete muitas chamadas. Quando um laço resolve melhor, use o laço.

Exemplo guiado 1: cálculo de frete

Retorno, parâmetro padrão e argumento nomeado

DESAFIO Crie `calcular_frete(peso, valor_compra, desconto=0.0)`: R$ 5,00 por kg e frete grátis em compras a partir de R$ 300.

frete.py

```
def calcular_frete(peso, valor_compra, desconto=0.0):
    if valor_compra >= 300:
        return 0.0
    frete = peso * 5
    return frete * (1 - desconto)

print(calcular_frete(4, 120))
print(calcular_frete(12, 120, desconto=0.2))
print(calcular_frete(3, 350))
```

SAÍDA

```
20.0
48.0
0.0
```

Exemplo guiado 2: resumo de notas

*args para as notas e **kwargs para as opções

DESAFIO Crie `resumo(*notas, **opcoes)` que devolve a média arredondada, a maior e a menor nota. A opção `casas` define o arredondamento.

resumo.py

```
def resumo(*notas, **opcoes):
    media = sum(notas) / len(notas)
    casas = opcoes.get("casas", 1)
    return round(media, casas), max(notas), min(notas)

print(resumo(7, 9, 6.5))
print(resumo(8, 10, casas=2))
```

SAÍDA

```
(7.5, 9, 6.5)
(9.0, 10, 8)
```

Exemplo guiado 3: registro com decorator

Um mini-Registry: o decorator guarda a função em um dicionário

registro.py

```
COMANDOS = {}

def comando(nome):
    def registrar(funcao):
        COMANDOS[nome] = funcao
        return funcao
    return registrar

@comando("oi")
def saudar():
    return "Olá!"

@comando("tchau")
def despedir():
    return "Até logo!"

print(COMANDOS["oi"](), COMANDOS["tchau"]())
print(sorted(COMANDOS))
```

SAÍDA

```
Olá! Até logo!
['oi', 'tchau']
```

Erros comuns com funções

Reconhecer a mensagem poupa tempo de depuração

Sintoma	Causa provável	Como corrigir
TypeError: missing argument	Faltou argumento na chamada	Passe todos ou defina um valor padrão
Resultado `None`	Usou print() no lugar de return	Troque por `return valor`
UnboundLocalError	Alterou uma global sem declará-la	Use parâmetro e `return`
Lista que acumula	`def f(x, lista=[])` reaproveita a lista	Use `None` e crie a lista dentro
RecursionError	Faltou o caso base	Garanta que o caso base é alcançado

Atividades da Aula 7

Módulo 4: prática guiada e projeto Registry (N1)

- **Prática guiada em sala**
  Refaça frete, resumo de notas e registro com decorator trocando os dados
- **Projeto Registry Pattern (N1) – Atividade 2**
  Sistema de Relatórios: gerador de relatórios sem cadeia de if/elif
- **Projeto Registry Pattern (N1) – Atividade 3**
  Pipeline de validação de formulário com decorator @registrar_validador
- **Preparação para o checkpoint de Funções (7 questões)**
  Retorno, parâmetros padrão, escopo, *args/**kwargs, callbacks, recursividade e docstrings

GIT Enunciados, instruções de execução e de entrega: **github.com/Hygo/python-fullstack-iteam-2sem**

Obrigado(a)!

Próxima aula: Módulos, pacotes e bibliotecas padrão
ITEAM – Instituto Tecnológico Educacional da Amazônia